
TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI
UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC

FICHA PRÁTICA 2**MODEL BINDING**

1. Objetivos

- Compreender a diferença prática entre GET e POST.
- Criar formulários HTML funcionais.
- Implementar Actions GET e POST.
- Aplicar Model Binding com parâmetros simples.
- Aplicar Model Binding com objetos complexos.
- Utilizar Debug para observar o processo de binding.
- Diagnosticar erros típicos de binding (nomes, tipos e fontes de dados).

2. Plano de Trabalho (gestão do tempo)

Sugestão de distribuição temporal (aprox.):

- Parte 1 (Revisão e Preparação): 10 min
- Parte 2 (GET com parâmetros): 15 min
- Parte 3 (Formulário POST + binding por parâmetros): 25 min
- Parte 4 (Binding por objeto + resultados): 25 min
- Parte 5 (Debug + erros intencionais): 15 min
- Exercício Final Integrador: 15 min
- Exercícios Extra (extensão): 15 min (ou mais, conforme ritmo)

3. Parte 1 – Revisão e Preparação (10 min)

Abrir o projeto criado na Aula 1 (MvcSimplesAula1).

Confirmar que o Controller CursoController está funcional.

Executar a aplicação e testar /Curso/Index.

4. Parte 2 – Action GET com Parâmetros (15 min)

Criar Action chamada Ola no CursoController:

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

```
public IActionResult Ola(string nome)
{
    return Content($"Olá {nome}");
}
```

Testar no browser: /Curso/Ola?nome=Ana

Checkpoint: A página deve mostrar 'Olá Ana'.

Extensão rápida: se nome for null ou vazio, devolver 'Olá visitante'.

5. Parte 3 – Formulário POST + Binding por Parâmetros (25 min)

Criar nova Action GET para mostrar o formulário:

```
[HttpGet]
public IActionResult Criar()
{
    return View();
}
```

Criar View Views/Curso/Criar.cshtml com o seguinte conteúdo:

```
<form method="post" asp-action="Criar">
    <div>
        <label>Nome:</label>
        <input type="text" name="Nome" />
    </div>
    <div>
        <label>Idade:</label>
        <input type="number" name="Idade" />
    </div>
    <button type="submit">Enviar</button>
</form>
```

Criar Action POST correspondente (binding por parâmetros simples):

```
[HttpPost]
public IActionResult Criar(string nome, int idade)
{
    return Content($"Nome: {nome}, Idade: {idade}");
}
```

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

Checkpoint: Submeter o formulário e verificar se os dados aparecem corretamente.

6. Parte 4 – Model Binding com Objeto (25 min)

Criar a classe Pessoa (em Models/Pessoa.cs ou numa pasta Models):

```
public class Pessoa
{
    public string Nome { get; set; }
    public int Idade { get; set; }
}
```

Alterar Action POST para receber o objeto Pessoa:

```
[HttpPost]
public IActionResult Criar(Pessoa pessoa)
{
    return Content($"Nome: {pessoa.Nome}, Idade: {pessoa.Idade}");
}
```

Checkpoint: Confirmar que o objeto é preenchido automaticamente.

Nota: o preenchimento depende de os inputs terem name="Nome" e name="Idade".

7. Parte 5 – Debug do Model Binding + Erros Intencionais (15 min)

Colocar breakpoint na Action Criar(Pessoa pessoa).

Executar em modo Debug (F5).

Inspecionar:

- Request.Form
- ModelState
- pessoa.Nome e pessoa.Idade

Experiência 1 (erro por nome): Alterar name="Nome" para name="NomeCompleto" e observar o resultado.

Experiência 2 (erro por tipo): escrever texto no campo Idade (se o browser permitir) e observar o binding.

Pergunta: Em que casos ModelState fica inválido? (Resposta será consolidada na Aula 3.)

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

8. Exercício Final Integrador (15 min)

Criar novo formulário com campos: Nome, Idade, Email.

Regras:

- Se idade < 18 mostrar mensagem 'Menor de idade'.
- Mostrar todos os dados introduzidos na resposta.

Sugestão: criar um novo model PessoaContacto com Nome, Idade e Email.

9. Exercícios Extra (Extensão após o exercício final)

Estes exercícios destinam-se a ocupar o tempo restante e consolidar o domínio do binding em cenários realistas.

EXTRA 1 – BINDING VIA ROUTE (10–15 MIN)

Criar uma Action que recebe parâmetros pela rota (route parameters) em vez da query string.

Exemplo de URL pretendido: /Curso/OlaRota/Ana

Implementação sugerida (atributo de rota):

```
[HttpGet("Curso/OlaRota/{nome}")]
public IActionResult OlaRota(string nome)
{
    return Content($"Olá {nome} (via rota)");
}
```

Checkpoint: testar /Curso/OlaRota/Ana

EXTRA 2 – FONTES DE BINDING: QUERY VS FORM (10–15 MIN)

Criar uma Action GET que recebe Nome pela query string e uma Action POST que recebe Nome pelo form.

Objetivo: perceber de onde vêm os valores (Value Providers).

Tarefa: escrever 2 URLs de teste e indicar onde o valor foi lido (QueryString vs Form).

EXTRA 3 – RESULTADO COM VIEW TIPADA (15–20 MIN)

Em vez de devolver Content(...), criar uma View Resultado.cshtml fortemente tipada.

Passos:

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

- Criar View: Views/Curso/Resultado.cshtml
- No POST, devolver: return View("Resultado", pessoa);
- Na View, usar @model Pessoa (ou PessoaContacto) e apresentar os campos.

Checkpoint: confirmar que a View recebe o modelo e o renderiza.

EXTRA 4 – PADRÃO GET/POST COM REDIRECT (PRG) (15–20 MIN)

Implementar o padrão Post/Redirect/Get para evitar re-submissão ao recarregar a página.

Objetivo: após POST, redirecionar para uma Action GET de resultado.

Implementação sugerida:

[HttpPost]

```
public IActionResult Criar(Pessoa pessoa)
```

```
{
```

```
    // guardar temporariamente (sem BD) para demonstração
```

```
    TempData["Nome"] = pessoa.Nome;
```

```
    TempData["Idade"] = pessoa.Idade.ToString();
```

```
    return RedirectToAction("Resultado");
```

```
}
```

[HttpGet]

```
public IActionResult Resultado()
```

```
{
```

```
    ViewData["Nome"] = TempData["Nome"];
```

```
    ViewData["Idade"] = TempData["Idade"];
```

```
    return View();
```

```
}
```

Checkpoint: submeter, ser redirecionado para /Curso/Resultado e conseguir ver os dados.

EXTRA 5 – ANÁLISE NO DEVTOOLS (10 MIN)

Abrir DevTools (F12) → Network.

Tarefa: comparar um pedido GET e um POST.

Registar por escrito:

- URL completo do GET (com query string)
- Local onde o POST transporta os dados (Request Payload / Form Data)

TÉCNICO PROGRAMADOR- INFORMÁTICA – 2TPI**UFCD 10792 - PROGRAMAÇÃO ASP.NET CORE MVC**

- Status code recebido (200/302)

10. Reflexão Final (10 min)

Responder por escrito:

1. 1) O que é o Model Binding?
2. 2) Qual a diferença entre GET e POST (incluindo implicações práticas)?
3. 3) Porque é importante o atributo name nos inputs?
4. 4) O que é o padrão PRG (Post/Redirect/Get) e que problema resolve?